Injection Function and Constructor injection

- In the angular 14 we can use the injection function instead of Construction injection

1- Import injection from Angular core

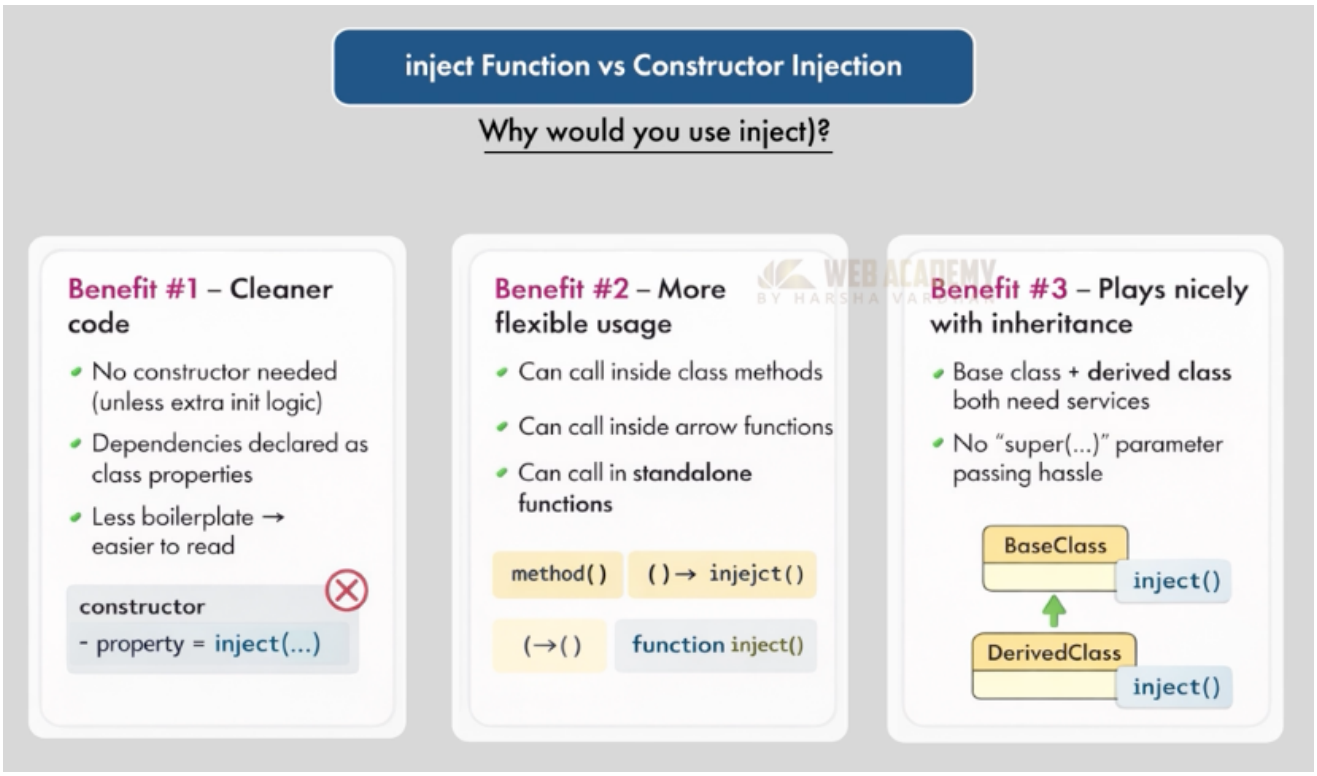
```
import { Component,inject } from '@angular/core';
```

```
private counterService : CounterServic = inject(CounterServic);
```

counter.ts | counter-display.ts

src > app > counter-display > counter-display.ts > ...

```
import { Component,inject } from '@angular/core';
import { CounterServic } from '../services/counter';

@Component({
  selector: 'app-counter-display',
  imports: [],
  templateUrl: './counter-display.html',
  styleUrl: './counter-display.css',
})
export class CounterDisplay {

  private counterService : CounterServic = inject(CounterServic);

  // constructor(private counterService: CounterServic){}

  getCurrentCount(): number{
    return this.counterService.getCount();
  }

}
```



inject Function vs Constructor Injection

Which one should you use? (Recommendation)

NEW PROJECT — New Project / New Component

Use inject()

- Modern approach (Angular 14+)
- Cleaner code
- More flexibility

```
private counterService = inject(
 CounterService);
```

EXISTING PROJECT — Existing Codebase

Keep constructor injection

- No need to refactor
- Works identically
- Prefer consistency / team preference

```
constructor(private counterService:
  CounterService) {
```


Both approaches can coexist in the same app

- In this course: you'll see **both**
- Task Manager project: **inject ()** used more often

inject Function vs Constructor Injection

Wrap-up & What's Next

Key Takeaways

- **inject** is a modern alternative to constructor injection
- Both approaches do the same thing
- **inject** = cleaner syntax + more flexibility

You now understand...

- [ ] **Services** (what they are)
- [ ] Creating services
- [ ] Dependency Injection (DI)
- [ ] Sharing data between components
- [ ] Injector hierarchy (levels of providers)